RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,
Bengaluru - 560059, Karnataka, India

Go, change the world

DEPARTMENT OF
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Bengaluru – 560059

Real-Time AI Governance Audit Pipeline

By

Project Report

Submitted by

NAME OF THE CANDIDATE(S)	USN
Aditya Ranjan	**1RV23AI008**
Arindam Gupta	**1RV23AI021**
Gnanendra Naidu N	**1RV23AI035**

in partial fulfillment for the requirement of 7th Semester

Stream Processing and Analytics (AI372IA)

Under the Guidance of

Dr. S. Anupama Kumar
Associate Professor
Department of AIML
RV College of Engineering
Bengaluru–59

Dr. Vijayalakshmi M.N
Associate Professor
Department of AIML
RV College of Engineering
Bengaluru–59

Academic Year

2026 – 2027

RV COLLEGE OF ENGINEERING®

(Autonomous Institution Affiliated to Visvesvaraya Technological University, Belagavi)

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Bengaluru – 560059

CERTIFICATE

Certified that the project work titled **'Real-Time AI Governance Audit Pipeline'** is carried out by **Aditya Ranjan (1RV23AI008)**, **Arindam Gupta (1RV23AI021)**, and **Gnanendra Naidu N (1RV23AI035)**, who are bonafide students of RV College of Engineering, Bengaluru, in partial fulfillment of the curriculum requirement of 7th Semester **Stream Processing and Analytics** Experiential Learning during the academic year **2026–2027**. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report. The report has been approved as it satisfies the academic requirements in respect of experiential learning work prescribed by the institution.

Signature of Faculty In-charge	**HoD**
Dept. of AIML, RVCE	**Dept. of AIML, RVCE**

External Examination

Name of Examiners	**Signature with date**
1	
2	

ABSTRACT

This report presents the design, implementation, deployment evidence, and evaluation of a real-time AI governance audit pipeline. The system receives events from five producer domains, normalizes them into a versioned contract, evaluates sixteen governance rules with Kafka Streams, scores findings, persists evidence in PostgreSQL, and exposes reporting views through Tableau. Optional Bedrock-based Investigator and Evidence Reviewer agents support evidence-grounded analysis with bounded revision and human escalation. The report combines repository analysis with dated local and AWS evidence, including MSK, RDS, EKS, CloudWatch, Kafka-topic, and Tableau observations. Results include a labelled 72-event demonstration with 48 expected findings detected, measured recovery scenarios, and preserved audit traces. The implementation is presented as a reference architecture and does not claim legal conformity or general model accuracy.

RASHTREEYA SIKSHANA SAMITHI TRUST ®
RV
INSTITUTIONS

Contents

Abstract **ii**

List of Tables **v**

List of Figures **vi**

1 Introduction **1**
1.1 Project Description 1
1.2 Report Organization 1
1.3 System overview 1

2 Problem Definition **2**
2.1 Problem Statement 2

3 Objectives **3**
3.1 Primary Objectives 3

4 Tools and Techniques Used **4**
4.1 Tools 4
4.2 Techniques 4
4.2.1 Project pipeline and flow 5
4.2.2 Kafka features incorporated 5

5 Implementation **7**
5.1 Background Information 7
5.2 Research gap and project position 8
5.3 Implementation architecture 9
5.4 High-level architecture 9
5.5 Module responsibilities 9
5.6 Data flow 10
5.7 Data Sources and Contracts 10
5.8 Producer groups 10

LAKSHMEESHA SIKSHANA SAMITHI TRUST ® R V INSTITUTIONS

5.9 Canonical event contract 10
5.10 Topics and integration 10
5.11 Rule Engine and Framework Mapping 11
5.12 Executable controls 11
5.13 Finding record 11
5.14 Risk Scoring, Storage and Reporting 12
5.15 Risk scoring 12
5.16 Persistence model 12
5.17 Agent integration 12
5.18 Deployment and Operations 13
5.19 Local operation 13
5.20 AWS operation 13
5.21 Resilience and controls 13

6 Results and Discussion **14**
6.1 Verification coverage 14
6.2 Measured functional run 14
6.3 Agentic AI Evaluation and Limitations 16
6.4 Observed outcomes 16
6.5 Limitations 17
6.6 Discussion 18

7 Conclusion **19**

8 Future Enhancements **20**

Appendix: Cloud and Reporting Evidence **21**

Bibliography **22**

RASHTREEYA SIKSHANA SAMITHI TRUST ®
R V
INSTITUTIONS

List of Tables

4.1 Techniques used in the implemented governance pipeline. 5

5.1 Comparison of recent sources and the corresponding implementation decision. 8

6.1 Repository-reported local evaluation results. 14

RASHTREEYA SIKSHANA SAMITHI TRUST ®
RV
INSTITUTIONS

List of Figures

4.1 Implemented project pipeline from source events through Kafka Streams, scoring, persistence, and reporting. 5

5.1 High-level architecture derived from the repository services, deployment manifests, and documented event flow. 9
5.2 Captured health and accepted-event evidence for the five producer adapters. 11
5.3 Persisted finding evidence showing the triggering event, rule, scores, and timestamps. 12
5.4 Live tool trace for the Investigator and Evidence Reviewer. 13

6.1 Supplied Tableau-style governance dashboard reference showing findings, risk, framework associations, latency, and incident details. . 15
6.2 Supplied project explanation reference for the Isolation Forest scorer, deterministic rules, and cooperating Bedrock agents. 15
6.3 Pipeline benchmark evidence for the labelled functional run. 16
6.4 Automated tests and consumer-tail verification. 16
6.5 Fresh reviewed report with preserved evidence. 17
6.6 Saved correction, revision, and human-escalation example. 17

8.1 CloudWatch log group and MSK broker log streams. 21
8.2 Kafka broker logs showing stabilized consumer coordination. 21
8.3 Active provisioned three-broker MSK cluster. 21
8.4 Governance topics with six partitions and replication factor three. . . 21
8.5 RDS PostgreSQL configuration for durable evidence. 21
8.6 PostgreSQL checkpoint activity in service logs. 21
8.7 EKS worker-node configuration. 22
8.8 EKS control-plane metrics API messages requiring follow-up. 22
8.9 Tableau governance dashboard. 22
8.10 Tableau Desktop authoring view. 22

Chapter 1

INTRODUCTION

This chapter introduces the project context, describes the repository-based system under study, and explains how the report is organized.

1.1 Project Description

This report analyses the Git repository *AI-Governance-Audit-Pipeline*, a runnable reference implementation for collecting AI-governance evidence, detecting compliance conditions, scoring risk, persisting findings, and supporting evidence-grounded investigation and review. It follows the supplied SPA report format: formal chapters, numbered sections, source references, and an evidence appendix.

1.2 Report Organization

The analysis covers the source tree, schemas, rule catalogue, Kafka Streams processor, Python services, database schema, deployment files, evaluation material, and the dated evidence package. It does not claim that framework mappings establish legal conformity; the repository itself correctly treats them as evidence-routing decisions.

1.3 System overview

The system is a modular event-to-framework pipeline. Five independent adapters normalize source events into a canonical contract and publish to dedicated topics. A stateful Kafka Streams processor evaluates 16 rules, then a risk scorer and idempotent PostgreSQL sink complete the path. Tableau views expose operational and compliance reporting. Optional Amazon Bedrock Investigator and Evidence Reviewer agents add bounded, durable, advisory analysis. The governance framing follows contemporary risk-management guidance from NIST, ISO, the OECD, UNESCO, and the EU AI Act [2, 3, 4, 5, 6, 7].

Chapter 2

PROBLEM DEFINITION

This chapter defines the governance and engineering problem addressed by the pipeline and establishes the background needed to understand its relevance.

2.1 Problem Statement

AI systems generate evidence across identity, infrastructure, pipelines, model registries, and LLM gateways. Without a shared event contract and durable correlation, governance teams cannot reliably connect raw activity to controls, risk, evidence, and review outcomes. The project addresses this by combining source-specific adapters, event-time rules, risk scoring, persistence, and human-review boundaries. This emphasis on traceability and lifecycle controls is consistent with recent AI governance guidance [2, 4, 7, 9].

RASHTREEYA SIKSHANA SAMITHI TRUST ® RV INSTITUTIONS

Chapter 3

OBJECTIVES

This chapter states the primary objectives used to guide implementation and evaluate whether the project outcomes were achieved.

3.1 Primary Objectives

1. **Canonical event normalization:** Normalize five heterogeneous producer systems into one versioned governance event while retaining source identity, timestamps, actors, objects, and evidence metadata.
   Expected outcome: every accepted event follows a common contract and can be correlated across the pipeline.

2. **Executable governance detection:** Detect 16 governance controls using stateless predicates, event-time windows, and stateful joins.
   Expected outcome: configured violations produce deterministic, keyed findings with traceable rule identifiers.

3. **Persistent evidence and review identity:** Preserve event, finding, score, evidence, job, report, and review identity from ingestion through storage.
   Expected outcome: each finding can be reconstructed from durable records rather than transient logs.

4. **Explainable risk scoring:** Provide deterministic severity-frequency risk signals alongside an Isolation Forest comparison without replacing the rule baseline.
   Expected outcome: reviewers can distinguish the policy result from the comparative ML signal.

5. **Resilient and bounded operations:** Demonstrate replay, recovery, testing, traceability, and advisory agent review with human escalation when evidence remains incomplete.
   Expected outcome: documented workloads recover predictably and agent outputs remain reviewable.

RASHTREEYA SIKSHANA SAMITHI TRUST ® RV INSTITUTIONS

Chapter 4

TOOLS AND TECHNIQUES USED

This chapter identifies the software, infrastructure, and analytical techniques used in the project, with emphasis on the Kafka-based processing approach.

4.1 Tools

The project uses Python for adapters, scoring, evaluation, and agent workers; Java and Apache Kafka Streams for stateful stream processing; PostgreSQL for durable evidence; Docker Compose and Redpanda for local execution; Amazon MSK, RDS, EKS, CloudWatch, and Tableau for cloud deployment and reporting; Terraform and Kubernetes manifests for infrastructure automation; and XeLaTeX for this report. These tools correspond to the repository's service boundaries and deployment artefacts.

4.2 Techniques

The implementation applies the following techniques to keep findings traceable, reproducible, and operationally recoverable:

- schema-first event normalization and source-specific validation;
- event-time windows with grace periods for late records;
- stateful table-table and stream-table joins;
- deterministic rule-based detection and severity-frequency scoring;
- Isolation Forest comparison for an additional unsupervised signal;
- idempotent persistence, keyed findings, and independent consumer groups;
- evidence retrieval, structured agent validation, bounded revision, and human escalation.

Table 4.1 summarizes how the selected techniques are used in the implementation.

Technique	Application	Purpose
Schema-first processing	Versioned governance-event contract and mapper validation	Preserve provenance and prevent incompatible source payloads
Event-time processing	Five-minute windows with one-minute grace	Detect temporal conditions despite late or out-of-order events
Stateful joins	Approval/evaluation tables and promotion stream-table join	Combine related evidence before creating a finding
Risk scoring	Severity-frequency baseline plus Isolation Forest	Provide explainable baseline and comparative anomaly signal
Durable review	PostgreSQL upserts, evidence snapshots, and bounded agents	Make findings reproducible and review outcomes auditable

Table 4.1: Techniques used in the implemented governance pipeline.

4.2.1 Project pipeline and flow

Figure 4.1 summarizes the implemented path from source systems to governance reporting. Each producer adapter exposes an HTTP event endpoint, validates and maps source-specific input into the versioned governance-event contract, and publishes to its dedicated Kafka topic. Kafka Streams consumes the five topics, evaluates stateless predicates, applies event-time windows and joins, and emits keyed compliance findings. The risk scorer adds a deterministic baseline and an online Isolation Forest comparison. The PostgreSQL sink persists events, findings, evidence, and scores, while Tableau and the optional agents read durable outputs.

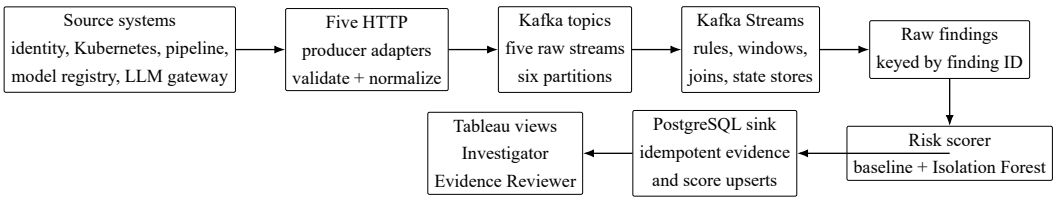


Figure 4.1: Implemented project pipeline from source events through Kafka Streams, scoring, persistence, and reporting.

4.2.2 Kafka features incorporated

The project uses Kafka as an integration and recovery boundary rather than as a direct service-to-service call graph. The five producer topics isolate source contracts and

allow each adapter to evolve independently. Keys are selected for the required state operation: actor IDs for failed-login windows and model keys for approval, evaluation, and promotion joins. The topology uses an event timestamp extractor so late or out-of-order records are evaluated against event time. A five-minute window with a one-minute grace period detects the fifth failed login without treating arrival time as the business event time.

Kafka Streams state stores materialize the model-approval state, model-evaluation state, combined model-governance evidence, and the failed-login window. The topology uses a table-table outer join to combine approval and evaluation evidence, then a stream-table left join to check a production promotion against that evidence. Stateless rule findings and stateful findings are merged into one raw-finding topic, and the finding ID becomes the output key for downstream correlation. JSON serdes keep the event and finding records typed while retaining a human-readable contract.

The deployment separates consumer groups for rule processing, risk scoring, persistence, and evaluation. This allows each downstream function to replay from its own committed offsets and prevents a reporting slowdown from blocking detection. The local Compose configuration creates six partitions per topic, runs two Kafka Streams threads, and uses Redpanda as the local Kafka-compatible broker. The AWS configuration raises durability through a three-broker MSK cluster, replication factor three, and minimum in-sync replica policy. These features support parallelism, replay, fault recovery, and independent scaling while keeping the audit trail reconstructable.

RASTREEYA SIKSHANA SAMITHI TRUST ® RV INSTITUTIONS

Chapter 5

IMPLEMENTATION

This chapter explains how the repository implements the architecture, contracts, rules, scoring, persistence, agent integration, deployment, and operational controls.

5.1 Background Information

Recent guidance treats AI governance as a lifecycle practice involving accountability, documentation, measurement, monitoring, and human oversight rather than a one-time model check [2, 3, 4, 5, 6, 7, 8]. Security guidance also highlights prompt injection, data leakage, excessive agency, and supply-chain risks in LLM-enabled systems [9, 10, 11]. The project responds to these themes through immutable evidence, explicit contracts, bounded agent loops, separate service identities, and a human-review terminal state.

RASHTREEYA SIKSHANA SAMITHI TRUST ® RV INSTITUTIONS

Source	Year	Main contribution	Design implication for this project
NIST AI RMF [2]	2023	Govern, map, measure, and manage AI risk	Rule catalogue, evidence, scores, and review states
NIST GenAI Profile [3]	2024	GenAI-specific risk actions and lifecycle profile	Bedrock agent boundaries and evidence grounding
ISO/IEC 42001 [4]	2023	AI management-system requirements and continual improvement	Durable controls, ownership, audit history
EU AI Act [7]	2024	Risk-tiered obligations, transparency, and governance	Framework labels and human escalation
OWASP LLM Top 10 [9]	2025	Common LLM application security risks	Prompt isolation, structured tools, bounded execution
Kafka Streams documentation [12]	2024	Event-time state, joins, and processing guarantees	Windowed failed-login detection and promotion joins
Kubernetes documentation [14]	2024	Probes, scaling, disruption budgets, and workload operation	Resilient service deployment and recovery checks
RAG survey literature [19]	2024	Retrieval grounding and evaluation concerns	Evidence snapshots and citation validation

Table 5.1: Comparison of recent sources and the corresponding implementation decision.

5.2 Research gap and project position

The literature provides principles, controls, and security taxonomies, but governance teams still need an operational evidence path that connects source activity to a finding, score, stored record, and review outcome. This repository occupies that implementation gap. Its contribution is systems integration and measurable traceability, not a new foundation model or a claim of legal certification.

5.3 Implementation architecture

5.4 High-level architecture

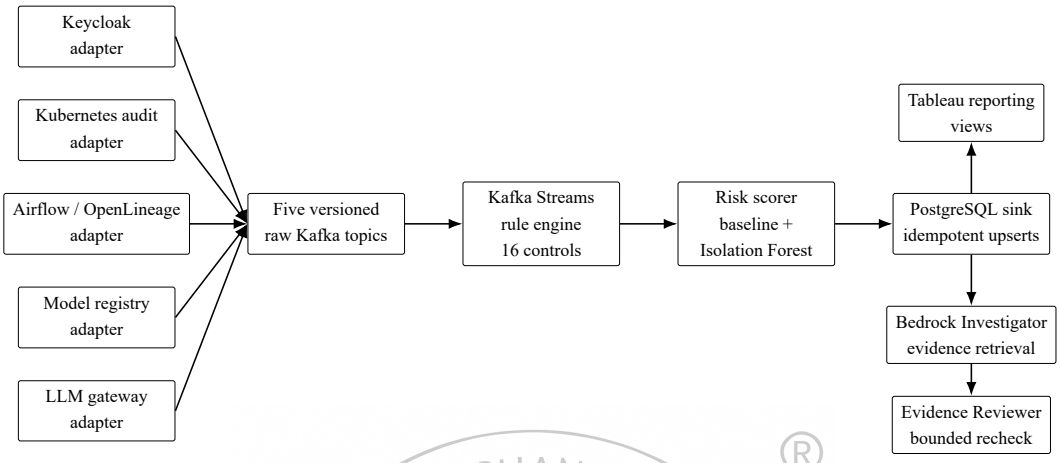


Figure 5.1: High-level architecture derived from the repository services, deployment manifests, and documented event flow.

The architecture separates producers, stream processing, scoring, storage, and reporting. Local Compose uses Redpanda and PostgreSQL; AWS deployment definitions provide MSK, EKS, RDS, ECR, IAM, VPC, and security groups. Kafka topics and JSON contracts are the primary integration boundary. The design follows documented event-stream processing and cloud-native operational patterns [12, 13, 14, 15].

5.5 Module responsibilities

Component	Responsibility
Producer adapters	Five standalone HTTP services for Keycloak, Kubernetes audit, Airflow/OpenLineage, model registry, and LLM gateway.
Stream processor	Java Kafka Streams topology with event-time windows, joins, rule catalog, and finding creation.
Risk scorer	Online Isolation Forest comparison with a deterministic severity-frequency baseline.
PostgreSQL sink	Idempotent event and finding upserts plus indexed evidence and reporting views.

Agents	Bedrock Investigator and Evidence Reviewer with durable jobs, structured contracts, and bounded revision.
Deployment	Docker Compose, Kubernetes manifests, Terraform infrastructure, and Tableau setup.

5.6 Data flow

```
source -> adapter -> raw topic -> canonical event -> rule engine
       -> finding -> risk scorer -> PostgreSQL -> Tableau / agents
```

5.7 Data Sources and Contracts

5.8 Producer groups

Each producer has its own adapter, health/readiness/metrics endpoints, image, deployment, scaling policy, IAM role, and topic. This reduces coupling and permits source-specific validation while maintaining a common downstream contract.

5.9 Canonical event contract

The versioned governance-event schema captures identity, source, timestamps, actor, object, action, attributes, and evidence metadata. Mappers remove source-specific variation while preserving useful provenance. Safe dead-letter handling prevents malformed input from silently entering the main path. This is aligned with current software supply-chain and observability practice [16, 17].

5.10 Topics and integration

The repository defines one raw topic per producer and separate consumer groups for rule processing, scoring, persistence, and evaluation. Six partitions and independent consumer groups support parallelism; offsets, keys, and event IDs enable reconciliation.

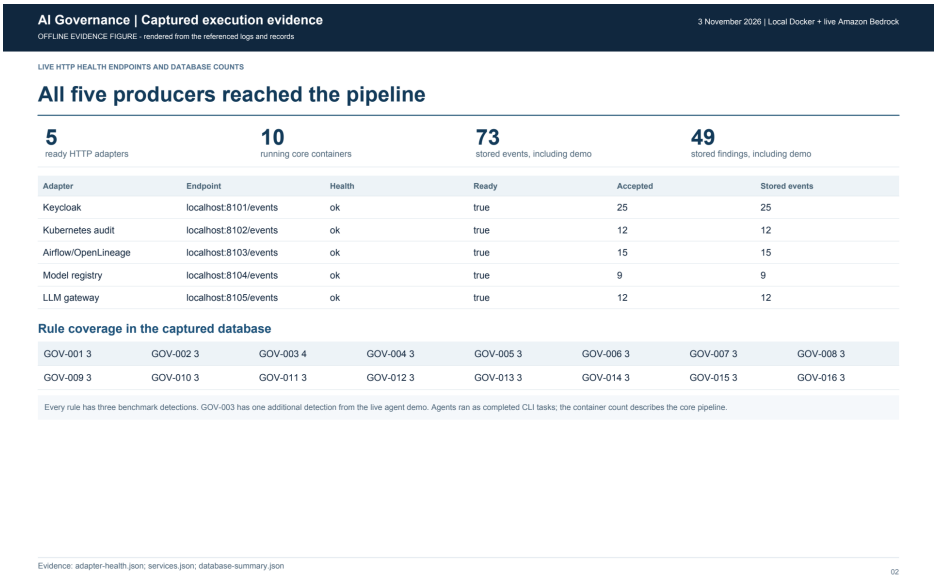


Figure 5.2: Captured health and accepted-event evidence for the five producer adapters.

5.11 Rule Engine and Framework Mapping

5.12 Executable controls

The 16-rule catalogue in `config/rules.yml` maps each control to NIST AI RMF 1.0, ISO/IEC 42001:2023, EU AI Act, NIST SP 800-53 Rev. 5, and GDPR. The Java implementation contains stateless predicates, a five-minute failed-login window with one-minute grace, and stateful promotion joins. The separation between a control mapping and a legal conclusion follows the risk-management approach in [2, 4, 7].

5.13 Finding record

Findings retain rule ID, severity, triggering event identity, timestamps, framework labels, evidence, and processing metadata. This makes a control result inspectable rather than a transient alert.

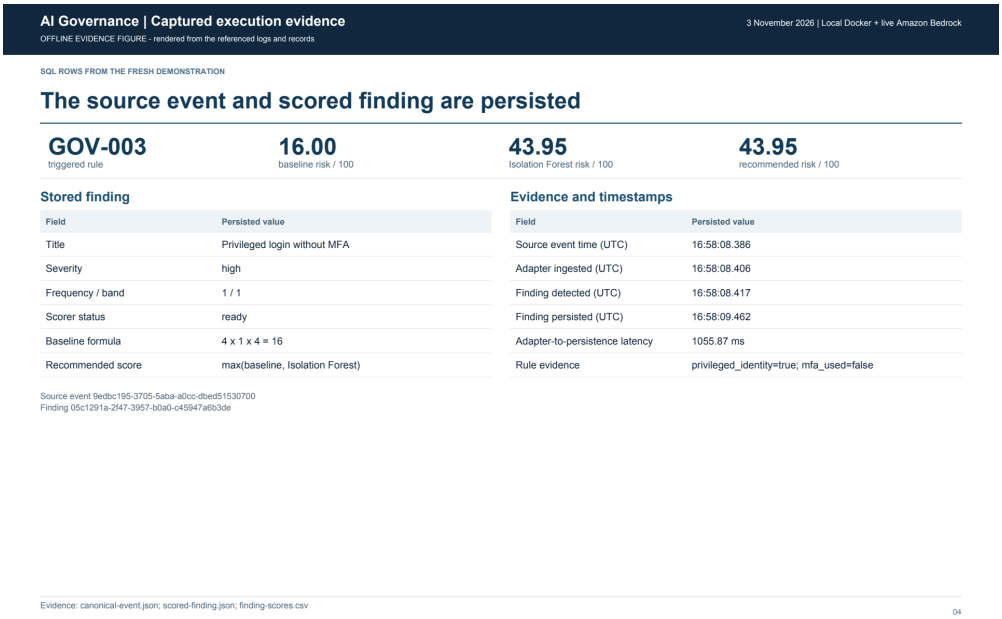


Figure 5.3: Persisted finding evidence showing the triggering event, rule, scores, and timestamps.

5.14 Risk Scoring, Storage and Reporting

5.15 Risk scoring

The deterministic baseline combines a 1–5 frequency bucket with severity and produces a normalized 0–100 score. An online Isolation Forest maintains bounded history and stores its score, delta, readiness status, and recommended score beside the baseline. The ML signal is comparative and does not remove the explainability of the rule result. The implementation uses the established scikit-learn API and keeps the baseline visible for auditability [18].

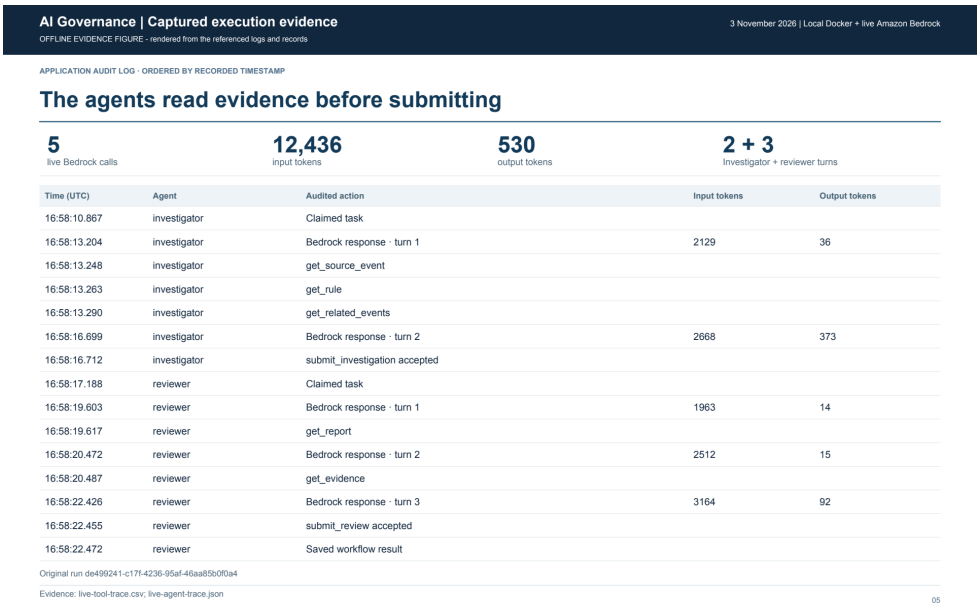
5.16 Persistence model

PostgreSQL stores raw events, findings, evidence, risk scores, jobs, agent reports, reviews, and audit history. Upserts and indexes support idempotency and case retrieval. Tableau views cover framework status, risk trend, incident history, and pipeline latency.

5.17 Agent integration

The Investigator retrieves eligible evidence and writes a structured report. The Evidence Reviewer checks citations and completeness, can request at most one revision, and may terminate in human review. Reports and evidence snapshots remain

immutable; agents do not remediate systems. This bounded pattern reflects current agent observability and responsible-agent guidance [21, 22, 20].



AI Governance | Captured execution evidence

OFFLINE EVIDENCE FIGURE - rendered from the referenced logs and records

3 November 2026 | Local Docker + live Amazon Bedrock

APPLICATION AUDIT LOG · ORDERED BY RECORDED TIMESTAMP

The agents read evidence before submitting

5	12,436	530	2 + 3
live Bedrock calls	input tokens	output tokens	Investigator + reviewer turns

Time (UTC)	Agent	Audited action	Input tokens	Output tokens
16:58:10.867	investigator	Claimed task		
16:58:13.204	investigator	Bedrock response · turn 1	2129	36
16:58:13.248	investigator	get_source_event		
16:58:13.263	investigator	get_rule		
16:58:13.290	investigator	get_related_events		
16:58:16.699	investigator	Bedrock response · turn 2	2668	373
16:58:16.712	investigator	submit_investigation accepted		
16:58:17.188	reviewer	Claimed task		
16:58:19.603	reviewer	Bedrock response · turn 1	1963	14
16:58:19.617	reviewer	get_report		
16:58:20.472	reviewer	Bedrock response · turn 2	2512	15
16:58:20.487	reviewer	get_evidence		
16:58:22.426	reviewer	Bedrock response · turn 3	3164	92
16:58:22.455	reviewer	submit_review accepted		
16:58:22.472	reviewer	Saved workflow result		

Original run de499241-c17f-4236-95af-46aa85b0f0a4

Evidence: live-tool-trace.csv; live-agent-trace.json

05

Figure 5.4: Live tool trace for the Investigator and Evidence Reviewer.

5.18 Deployment and Operations

5.19 Local operation

The documented local path uses Docker Desktop with at least 6 GB available: copy `.env.example`, start Compose, inject labelled evaluation events, and export a report. Redpanda stands in for MSK and a PostgreSQL container stands in for RDS.

5.20 AWS operation

Terraform provisions a VPC, three-broker MSK, EKS, RDS, ECR, security groups, and workload IAM. Kubernetes manifests define services, probes, HPAs, PDBs, and consumer scaling. The deployment instructions correctly require image publication, secret creation, placeholder replacement, and an evaluation before recording results.

5.21 Resilience and controls

Idempotent writes, retrying producers, replication factor three, minimum ISR two, consumer groups, and reconciliation checks address loss and restart scenarios. Operational scripts pause and resume services; failure procedures distinguish the local single-broker demonstration from AWS broker failover.

Chapter 6

RESULTS AND DISCUSSION

This chapter presents the observed functional, resilience, cloud, dashboard, and agent-evaluation evidence, followed by interpretation and limitations.

6.1 Verification coverage

The repository includes Python, Java, mapper, integration, settings, reviewer, topology, scorer, and evaluation tests. Requirements traceability links each major requirement to implementation and verification. The 3 November evidence package records 83 agent tests passed in 9.65 seconds.

6.2 Measured functional run

Scenario	Events	Exp./Obs.	Precision	Recall	Throughput
Baseline	72	48/48	1.0000	1.0000	130.90/s
Processor replacement	72	48/48	1.0000	1.0000	11.50/s
Broker restart	72	48/48	1.0000	1.0000	11.48/s
Independent adapters	72	48/48	1.0000	1.0000	667.12/s

Table 6.1: Repository-reported local evaluation results.

The fresh 3 November evidence also reports 72 synthetic source-shaped events, 48 expected findings across all 16 rules, p50 353.84 ms, p95 1,878.88 ms, p99 2,181.69 ms, and 39.04 events/s over the observed persistence interval. These are small-workload measurements, not general model-accuracy claims. The project dashboard in Figure 6.1 presents the same reporting concepts in a compact view, while Figure 6.2 summarizes how the deterministic rule engine, Isolation Forest scorer, and Bedrock agents work together.

RASHTREEYA SIKSHANA SAMITHI TRUST ® RV INSTITUTIONS

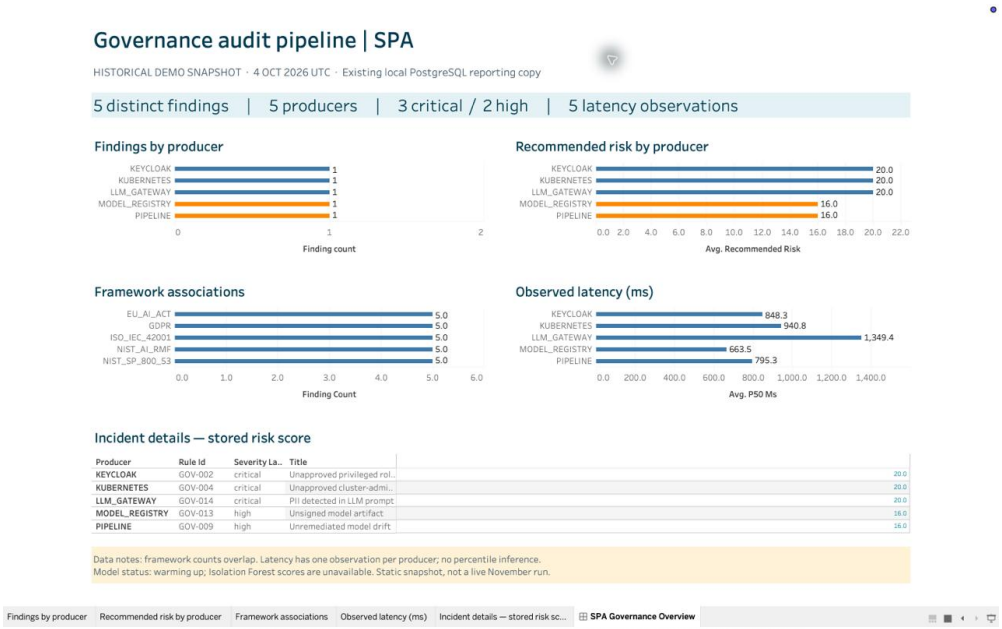


Figure 6.1: Supplied Tableau-style governance dashboard reference showing findings, risk, framework associations, latency, and incident details.

Yes—the project includes both traditional machine learning and LLM-powered agents.

Component	Technology	Role
Risk scorer	**Isolation Forest**, using scikit-learn	Scores how unusual a governance finding is
Investigator and Reviewer	**Amazon Bedrock LLM**	Investigate findings, review evidence and request corrections
Governance rule engine	16 deterministic rules	Detects configured policy violations

The **Isolation Forest** uses severity, frequency, baseline risk, time of day, rule ID and producer type. By default, it:

- Starts training after **32 findings**.
- Retrains every **25 additional findings**.
- Keeps up to **1,000 recent findings**.
- Sets the recommended risk to the higher of the baseline and ML scores.

This is implemented in scorer.py (line 17).

In our recorded demo, the **baseline score was 16**, the **ML score was 43.95**, and the recommended score became **43.95**.

For your presentation, you can describe it as: **"A governance pipeline combining rule-based detection, unsupervised ML risk scoring, and two cooperating AI agents."**

Figure 6.2: Supplied project explanation reference for the Isolation Forest scorer, deterministic rules, and cooperating Bedrock agents.

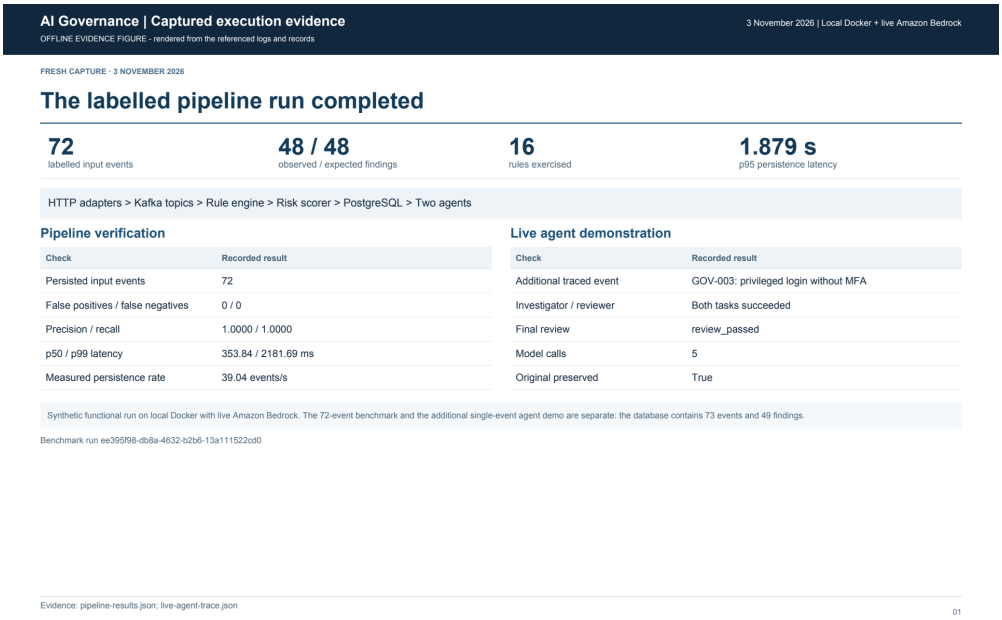


Figure 6.3: Pipeline benchmark evidence for the labelled functional run.

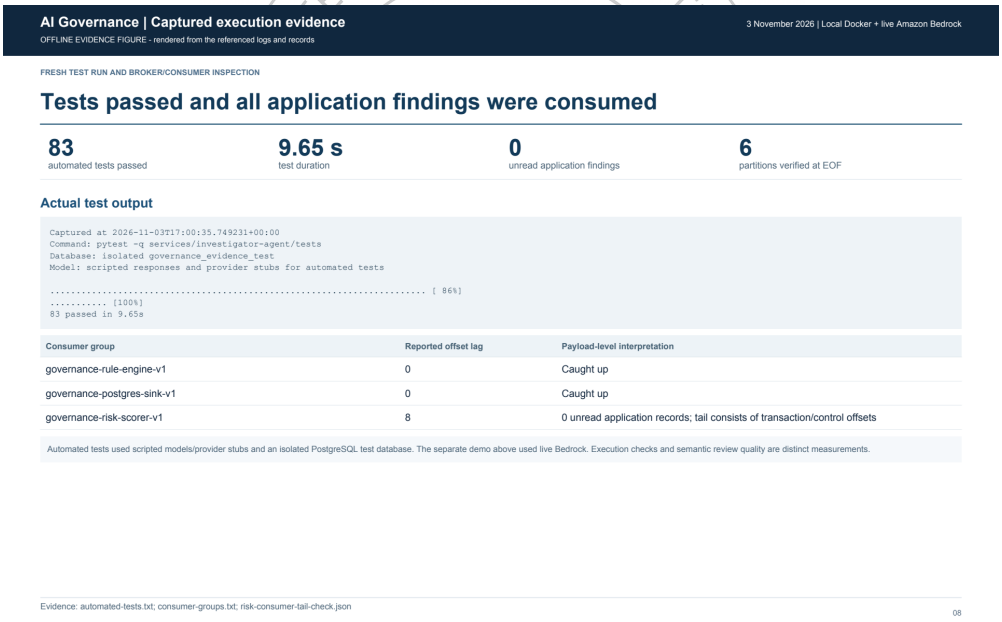


Figure 6.4: Automated tests and consumer-tail verification.

6.3 Agentic AI Evaluation and Limitations

6.4 Observed outcomes

One fresh individual case completed investigation and review with `review_passed`; the original report and evidence snapshot were preserved. A saved correction example demonstrates cited feedback, one revision, recheck, and terminal `needs_human_`

`review`. CloudWatch evidence for the selected window records five Bedrock invocations, 12,436 input tokens, 530 output tokens, and 1,476.4 ms average latency.

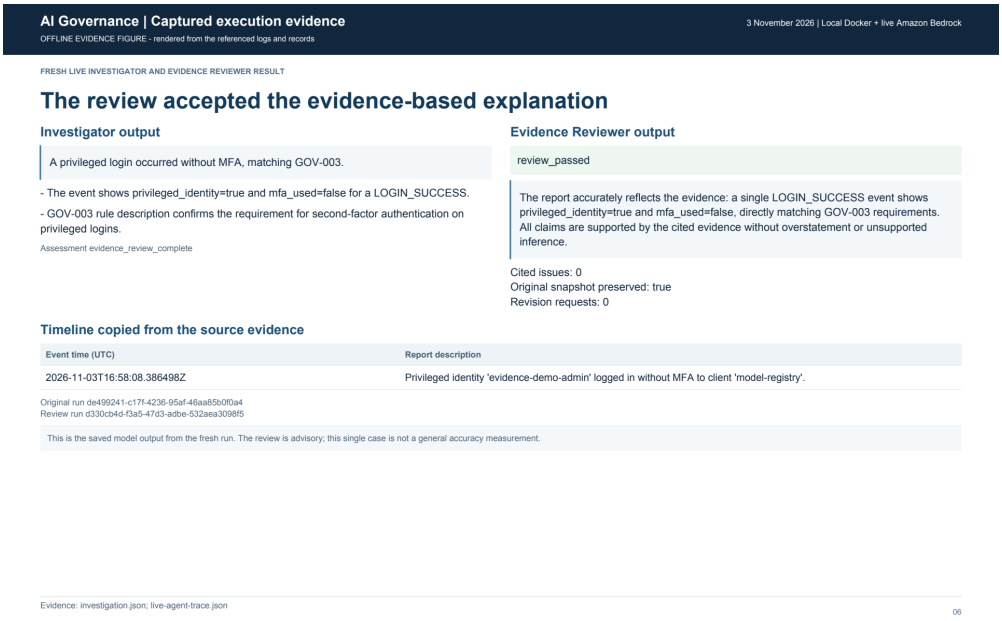


Figure 6.5: Fresh reviewed report with preserved evidence.

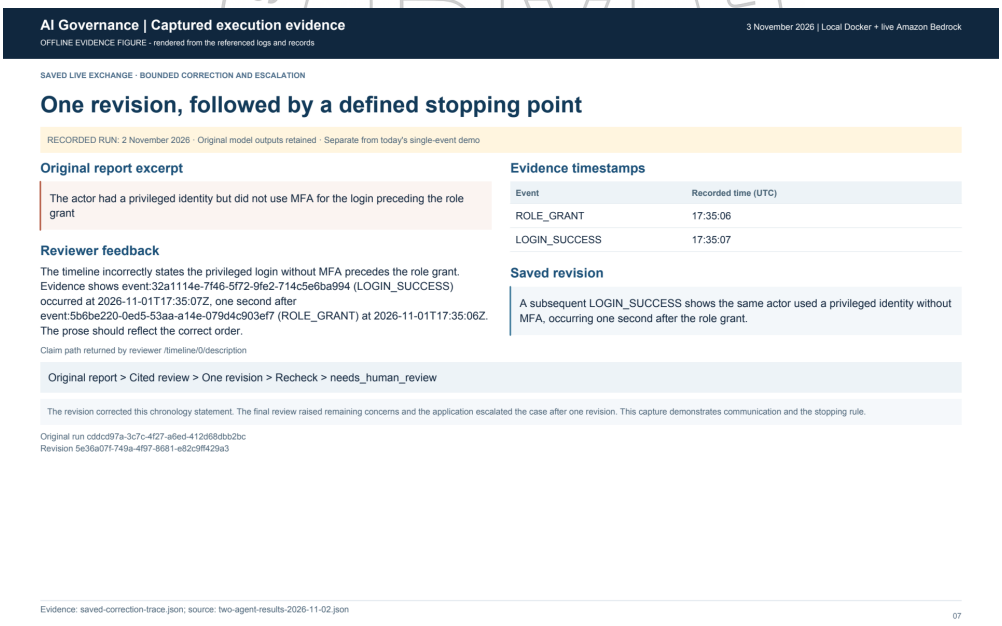


Figure 6.6: Saved correction, revision, and human-escalation example.

6.5 Limitations

The reviewer is advisory and must not be treated as an automatic approval gate. The evidence package contains selected demonstrations rather than corpus-wide semantic-quality evaluation. Framework labels are routing metadata, not legal conclusions. The

AWS metrics are account/model/region aggregates for the selected window, while the application trace provides case-level correlation.

6.6 Discussion

The measured results indicate that the implemented objectives were met for the documented workloads: the labelled pipeline recovered all expected findings in the baseline and fault-injection demonstrations, while the evidence package preserves the separate agent outcomes and their limitations. The AWS screenshots corroborate that the cloud resources were provisioned and observable, but they do not replace a controlled production readiness review. The EKS control-plane logs also show stale metrics API discovery messages, which should be treated as an operational follow-up rather than silently classified as a successful health result.

RASHTREEYA SIKSHANA SAMITHI TRUST ®
R V
INSTITUTIONS

Chapter 7

CONCLUSION

This chapter summarizes the engineering contribution, the evidence-supported outcomes, and the practical significance of the completed pipeline.

The project is a coherent, runnable governance-audit platform with unusually strong traceability for a reference implementation. Its strongest engineering decisions are independent adapters, explicit contracts, event-time/stateful processing, idempotent persistence, measured recovery tests, immutable evidence, and bounded agent workflows. The main adoption requirement is operational discipline: execute the documented evaluation in the target environment, retain evidence, and preserve human review for uncertain or corrected agent outcomes.

RASHTREEYA SIKSHANA SAMITHI TRUST ®
RV
INSTITUTIONS

Chapter 8

FUTURE ENHANCEMENTS

This chapter identifies extensions that would improve production readiness, evaluation depth, operational resilience, and governance coverage.

Future work should add a production-scale semantic-quality benchmark for the agents, formal schema compatibility checks across release versions, stronger alerting for stale Kubernetes metrics APIs, automated evidence retention policies, multi-region disaster recovery, continuous Tableau refresh validation, and a governed approval workflow for changes to framework mappings. A larger labelled corpus would also allow calibration of risk thresholds beyond the current demonstration workloads.

RASHTREEYA SIKSHANA SAMITHI TRUST ®
RV
INSTITUTIONS

APPENDIX: CLOUD AND REPORTING EVIDENCE

The following screenshots were supplied with the project analysis package and are included as operational evidence. Figures 8.1–8.8 document cloud infrastructure and service observability. Figures 8.9–8.10 document the Tableau reporting layer. The screenshots are descriptive evidence of the captured environment and should be read with the repository's dated provenance files.

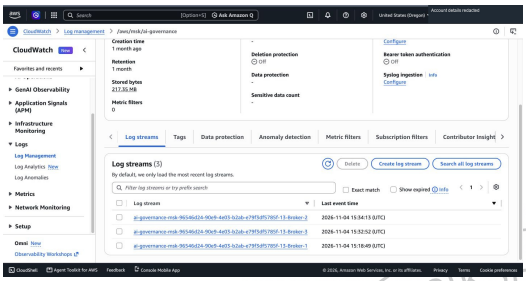


Figure 8.1: CloudWatch log group and MSK broker log streams.

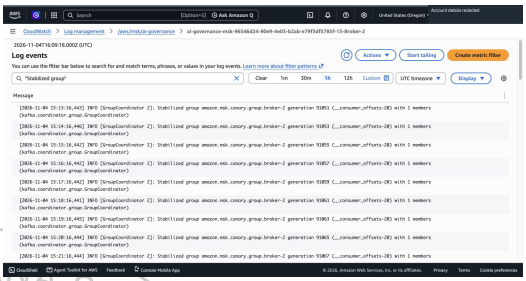


Figure 8.2: Kafka broker logs showing stabilized consumer coordination.

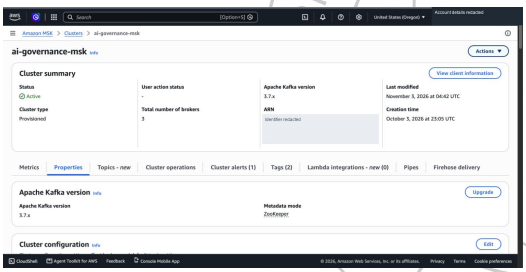


Figure 8.3: Active provisioned three-broker MSK cluster.

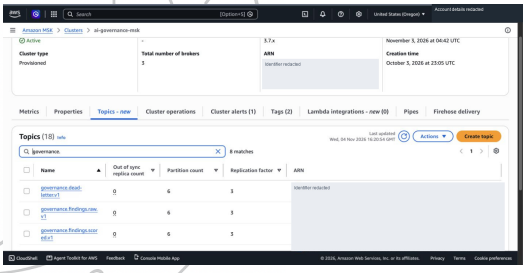


Figure 8.4: Governance topics with six partitions and replication factor three.

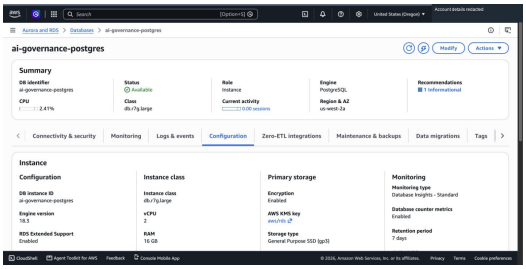


Figure 8.5: RDS PostgreSQL configuration for durable evidence.

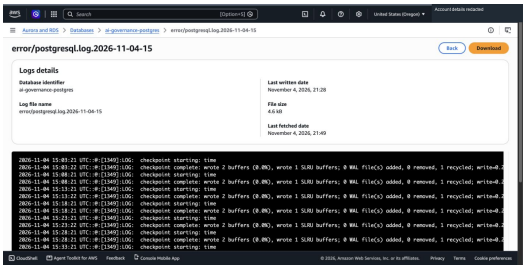


Figure 8.6: PostgreSQL checkpoint activity in service logs.

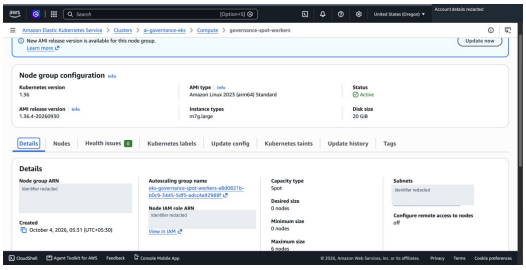


Figure 8.7: EKS worker-node configuration.

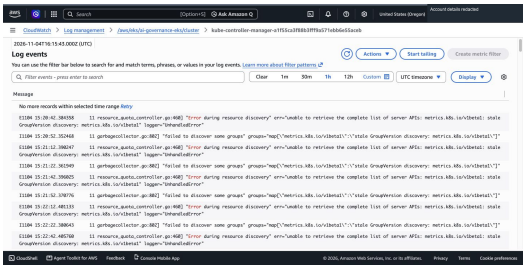


Figure 8.8: EKS control-plane metrics API messages requiring follow-up.

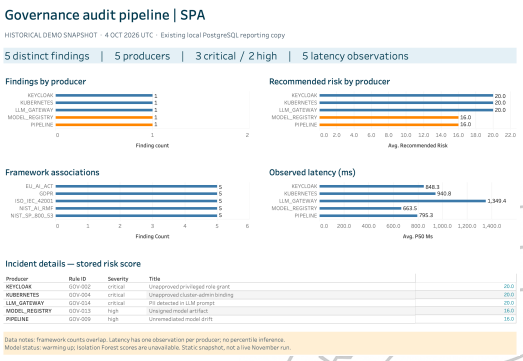


Figure 8.9: Tableau governance dashboard.

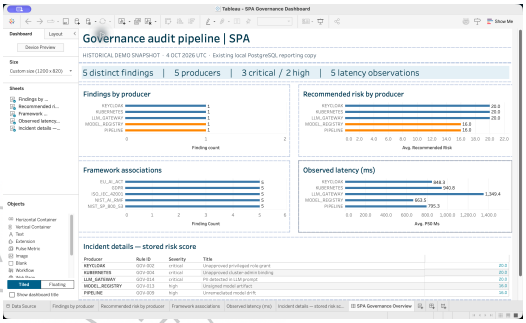


Figure 8.10: Tableau Desktop authoring view.

BIBLIOGRAPHY

[1] A. Ranjan, A. Gupta, and G. N. Naidu, "AI-Governance-Audit-Pipeline," GitHub repository, 2026. Available: https://github.com/Aditya-Ranjan1234/AI-Governance-Audit-Pipeline

[2] E. Tabassi, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," NIST AI 100-1, 2023. doi:10.6028/NIST.AI.100-1.

[3] NIST, "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile," NIST AI 600-1, 2024. Available: https://doi.org/10.6028/NIST.AI.600-1

[4] ISO/IEC, "ISO/IEC 42001:2023 Information technology – Artificial intelligence – Management system," 2023. Available: https://www.iso.org/standard/81230.html

[5] OECD, "OECD AI Principles," updated 2024. Available: https://oecd.ai/en/ai-principles

[6] UNESCO, "Recommendation on the Ethics of Artificial Intelligence," 2021. Available: https://unesdoc.unesco.org/ark:/48223/pf0000381137

[7] European Parliament and Council, "Regulation (EU) 2024/1689 (Artificial Intelligence Act)," 2024. Available: https://eur-lex.europa.eu/eli/reg/2024/1689

[8] Stanford Institute for Human-Centered Artificial Intelligence, "AI Index Report 2024," 2024. Available: https://aiindex.stanford.edu/report/

[9] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," 2025. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

[10] MITRE, "MITRE ATLAS: Adversarial Threat Landscape for Artificial-Intelligence Systems," 2024. Available: https://atlas.mitre.org/

[11] Google, "Secure AI Framework (SAIF)," 2023. Available: https://safety.google/cybersecurity-advancements/saif/

[12] Apache Software Foundation, "Kafka Streams Developer Guide," 2024. Available: https://kafka.apache.org/documentation/streams/

RASHTREEYA SIKSHANA SAMITHTI TRUST ® RV INSTITUTIONS

[13] Redpanda Data, "Redpanda Documentation," 2024. Available: https://docs.redpanda.com/current/

[14] Kubernetes Authors, "Kubernetes Documentation: Workloads, Probes, Scaling and Disruptions," 2024. Available: https://kubernetes.io/docs/

[15] HashiCorp, "Terraform AWS Provider Documentation," 2024. Available: https://registry.terraform.io/providers/hashicorp/aws/latest/docs

[16] OpenSSF, "Open Source Security Foundation Scorecard Documentation," 2024. Available: https://scorecard.dev/

[17] OpenTelemetry Authors, "OpenTelemetry Documentation," 2024. Available: https://opentelemetry.io/docs/

[18] scikit-learn Developers, "IsolationForest API Reference," version 1.5, 2024. Available: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html

[19] Y. Gao et al., "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv:2312.10997, revised 2024. Available: https://arxiv.org/abs/2312.10997

[20] M. Wang et al., "A Survey on Large Language Model based Autonomous Agents," arXiv:2308.11432, revised 2024. Available: https://arxiv.org/abs/2308.11432

[21] Amazon Web Services, "Amazon Bedrock User Guide: Agents," 2025. Available: https://docs.aws.amazon.com/bedrock/latest/userguide/agents.html

[22] Amazon Web Services, "Monitor Amazon Bedrock Agents using CloudWatch Metrics," 2025. Available: https://docs.aws.amazon.com/bedrock/latest/userguide/monitoring-agents-cw-metrics.html

ACHIEVEMENT THROUGH KNOWLEDGE SAMARTHI TRUST ® RV INSTITUTIONS